If the first group, the red alternative, matched, then the conditional subpattern is blue, otherwise it is red.

Named Subroutines

There's a trick that allows us to write subroutines inside regular expressions. Recall that we can invoke a named group using \g<name>, and we define the group using (?<name>...). Normally, the definition of the group is itself matched as part of executing the pattern. However, if you add the suffix {0} to the group, it means "zero matches of this group," so the group is not executed when first encountered.

In this example, we use that trick to name all our subgroups up front, and then use the named versions to build the final match:

tut_regexp/named_subroutines.rb

```
sentence = %r{
  (?<subject>   cat   | dog   | gerbil    ){0}
  (?<verb>      eats  | drinks| generates ){0}
  (?<object>    water | bones | PDFs      ){0}
  (?<adjective> big   | small | smelly    ){0}
  (?<opt_adj>   (\g<adjective>\s)?        ){0}

  The\s\g<opt_adj>\g<subject>\s\g<verb>\s\g<opt_adj>\g<object>
}x

md = sentence.match("The cat drinks water")
puts "The subject is #{md[:subject]} and the verb is #{md[:verb]}"

md = sentence.match("The big dog eats smelly bones")
puts "The last adjective in the second sentence is #{md[:adjective]}"

sentence =~ "The gerbil generates big PDFs"
puts "And the object in the last sentence is #{$~[:object]}"
```

produces:

```
The subject is cat and the verb is drinks
The last adjective in the second sentence is smelly
And the object in the last sentence is PDFs
```

Setting Options

We saw earlier that you can control the characters matched by \b, \d, \s, and \w (along with their negations). To do that, we embedded a sequence such as (?u) in our pattern. That sequence sets an option inside the regular expression engine.

We also saw at the start of this chapter that you can add one or more of the options i (case insensitive), m (multiline), and x (allow spaces) to the end of a regular expression literal. You can also set these options within the pattern itself. They are set using (?i), (?m), and (?x). You can also put a minus sign in front of these three options to disable them.

(?adimux)	Turns on the corresponding option. If used inside a group, the effect is limited to that group.
(?-imx)	Turns off the i, m, or x option.
(?adimux:re)	Turns on the option for _re_.
(?-imx:re)	Turns off the option for *re*.